

Technical Learning Report: Week 2 Day 1

- **Date:** September 7, 2026
- **Intern Name:** Shekhar Singh Rawal
- **Subject:** Test Case Design

1. Objective

The objective of the day was to understand **test case design and test case prioritisation**, including test case structure, pre-conditions, test steps, expected results, post-conditions, positive and negative test cases, coverage, requirement traceability, and prioritisation. The session also covered **STLC, bug life cycle, bug report elements, effective test case writing**. Practical test cases were designed for the Aaravya Clothing website add to cart feature with focus on connecting test cases to requirements rather than testing only individual UI fields.

2. Technical Breakdown

A. Test Case and Its Elements

A **test case** is a set of actions used to verify a particular feature or functionality of a software application.

The main elements studied were:

- **Pre-condition:** Conditions that must be true before the test can run.
- **Test Steps:** Actions performed during test execution.
- **Expected Result:** The result that should occur after performing the test steps.
- **Post-condition:** The state of the system after test execution.

Effective Test Case Writing

The following characteristics of effective test cases were studied:

- Clear and understandable.
- One clear purpose.
- Complete within the required scope.
- Traceable to a requirement.
- Repeatable and precise.
- Independent where possible.

B. Positive and Negative Test Cases

Positive Test Case

Checks whether the system works correctly when valid input is provided.

Example: Enter valid login credentials and verify successful login.

Negative Test Case

Checks whether the system correctly handles invalid input or conditions.

Example: Enter invalid login credentials and verify that the appropriate error is displayed.

C. Test Case Prioritisation

Test case prioritisation determines the order in which test cases should be executed.

Three approaches studied were:

1. Risk-Based Prioritisation

High-risk or business-critical functionality is tested with higher priority.

2. Coverage-Based Prioritisation

Test cases that provide greater coverage are given higher execution priority.

3. Requirement-Based Prioritisation

Test cases are prioritised according to the importance of their associated requirements.

D. Test Coverage

Test coverage refers to how much of the required functionality or testable area is covered by testing.

Coverage can include:

- Requirement coverage.
- Functional coverage.
- Code coverage.

E. Requirement Traceability

Requirement traceability maintains a connection between requirements and their corresponding test conditions, test cases, and defects.

The relationship studied was:

Requirement → Test Condition → Test Case → Defect

This helps verify that requirements are actually covered by testing.

F. Software Testing Life Cycle (STLC)

The major STLC stages studied were:

1. Requirement Analysis
2. Test Planning
3. Test Case Development
4. Test Environment Setup
5. Test Execution
6. Test Closure

STLC provides a structured sequence for carrying out testing activities.

G. Bug and Bug Life Cycle

A **bug** is a flaw, defect, error, or fault in software that may cause the system to behave differently from the expected result.

The bug life cycle studied was:

New → Open → Fixed → Retest → Closed

Other possible outcomes include **Rejected** and **Reopened**. A defect may also be deferred when it is postponed for a future release.

H. Bug Report Elements

The important elements of a bug report studied were:

- Bug ID
- Bug Title
- Description
- Severity
- Priority

- Steps to Reproduce
- Reported By
- Status
- Fixed By
- Closed On

Severity represents the impact of a defect, while **Priority** represents how urgently it should be addressed.

3. Practical Implementation

A. Benefit Store Test Case Design

Test cases were designed for the **Benefit Store application**, covering the scope up to **Add-to-Cart**.

The test cases included:

- Positive test cases.
- Negative test cases.
- Edge cases.
- Prioritisation.
- Requirement-based traceability.

Each test case was structured using the learned test case elements and linked to the relevant business rule or requirement.

The detailed test cases were maintained separately and are not reproduced in this report.

B. Test Case Prioritisation

The designed test cases were considered using:

- Risk-based prioritisation.
- Coverage-based prioritisation.
- Requirement-based prioritisation.

This helped establish how test execution can be organised according to risk, coverage, and requirement importance.

C. Requirement Traceability

Test cases were linked to their corresponding requirements or business rules.

The traceability flow used was:

Requirement → Test Condition → Test Case → Defect

D. STLC and Bug Life Cycle

The STLC stages and bug life cycle were reviewed and connected with the testing process.

STLC:

Requirement Analysis → Test Planning → Test Case Development → Environment Setup → Test Execution → Test Closure

Bug Life Cycle:

New → Open → Fixed → Retest → Closed

E. AI-Assisted Activity

A short AI-assisted activity was performed to support test case design.

AI was used to generate sample test cases for a login page using fields such as:

- Email
- Password
- Remember Me

The generated output was considered as a reference and compared with the understanding of structured test case design, including positive and negative scenarios, expected results, and requirement-based coverage.

4. Key Findings & Best Practices

- Test cases should verify requirements and business rules rather than focusing only on UI fields.
- Pre-conditions define what must be ready before execution.
- Test steps should clearly describe the actions performed.
- Expected results define the required system behavior.
- Positive and negative cases are both necessary for meaningful coverage.
- Test cases can be prioritised using risk, coverage, or requirement importance.
- Requirement traceability connects requirements with test cases and defects.

- STLC provides a structured approach to testing activities.
- Bug reports require enough information to reproduce and track defects.
- Severity and priority represent different aspects of a defect.
- Branch testing focuses on executing alternative program branches.
- Checklist-based testing uses predefined items or criteria to support systematic checking.
- AI-generated test cases should be reviewed against the tester's own understanding rather than being accepted without evaluation.

5. Conclusion

Day 1 of Week 2 focused on **test case design, prioritisation, coverage, and requirement traceability**. Practical test cases were prepared for the Aaravya Clothing website to the Add-to-Cart scope, including positive, negative, and edge cases.

The session also covered **STLC, bug life cycle, bug report elements, effective test case writing**. A short AI-assisted activity was also performed to review sample login test cases.

Overall, the day strengthened the understanding of how to create structured, prioritised, and traceable test cases that verify requirements and business rules.

Glossary

Term	Meaning
Test Case	A set of actions used to verify a particular feature or functionality.
Pre-condition	Condition that must be satisfied before test execution.
Expected Result	Result that should occur after performing the test steps.
Post-condition	State of the system after test execution.
Positive Testing	Testing with valid inputs or expected conditions.
Negative Testing	Testing invalid inputs or conditions to verify proper handling.
Test Coverage	Extent to which the required functionality or testable area is covered.
Traceability	Linking requirements with test conditions, test cases, and defects.
STLC	Software Testing Life Cycle.
Bug	A flaw or defect that causes unexpected system behavior.

